

Republic of Cameroon

Peace-Work-Fatherland

Academic year: 2025-2026

République du Cameroun

Paix-Travail-Patrie

Année académique: 2025-2026



PROJET DE PROGRAMMATION ORIENTÉE OBJETS(PYTHON)

SIMNet: SIMULATEUR DE RÉSEAU INTELLIGENT

GROUPE 03

Membres du groupe:

- ❖ AROUNG Ralph Stéphane Darrell
- ❖ ESSIE ABWA David Erwann
- ❖ GODSWILL CHONGSI Junior
- ❖ NGANKAK DZUKOU Louis-Georges
- ❖ TAMBO DJOUONTZO Stephen Darren

Classe : INGE 3 SRT

Sous la coordination de : M. FEDIM

Année académique : 2025-2026

SOMMAIRE

INTRODUCTION GÉNÉRALE ET CONTEXTE.....	2
I. ARCHITECTURE LOGICIELLE ET EXPLICATIONS DES CLASSES	3
1. Abstraction et Modélisation des Équipements	3
2. Gestion de la Topologie et Interconnexions.....	3
3. Simulation du trafic et algorithme de routage	4
4. Sécurité et filtrage	4
5. Surveillance et rapports	5
II. CONCEPTS POO COUVERTS.....	6
III. DIAGRAMME DE CLASSES COMPLET	7
IV. EXTRAITS DU CODE SOURCE	9
V. SCÉNARIOS DE CAS D'UTILISATION ET VALIDATION DU LOGICIEL ...	12
1. Scénario 1 : Initialisation de l'Architecture Réseau	12
2. Scénario 2 : Simulation de Routage Nominal	12
3. Scénario 3 : Comportement face aux Anomalies et Sécurité.....	12
VI. DIFFICULTÉS RENCONTRÉS ET SOLUTIONS APPORTÉES.....	13
CONCLUSION ET PERSPECTIVES.....	14
BIBLIOGRAPHIE.....	15

INTRODUCTION GÉNÉRALE ET CONTEXTE

Dans le cadre de l'unité d'enseignement de Programmation Orientée Objet (POO) en Python sous la supervision de M. Stephane Fedim, ce projet présente la conception et le développement de SIMNet. SIMNet est un simulateur de réseau d'entreprise complet retenant l'ensemble des concepts avancés de la POO (abstraction, héritage, encapsulation, et polymorphisme). Cet outil permet à un administrateur ou à un ingénieur système de modéliser une infrastructure topologique, d'y injecter du trafic de paquets de données et de tester de manière logicielle les politiques de filtrage de sécurité ainsi que la performance des équipements.

I. ARCHITECTURE LOGICIELLE ET EXPLICATIONS DES CLASSES

1. Abstraction et Modélisation des Équipements

L'ensemble du matériel physique réseau est architecturé autour d'une classe mère abstraite nommée `Equipement` (définie via le module `abc`). Elle encapsule les attributs privés communs tels que le nom, la marque, l'adresse IP et le statut d'activité, exposant des accesseurs (getters) et mutateurs (setters) sécurisés. Chaque type de matériel hérite de cette classe et implémente de manière polymorphique sa méthode abstraite `afficher_info()`.

- **Routeur** : Gère la table de routage, l'aiguillage inter-réseau et le calcul de chemins optimaux via des algorithmes de plus court chemin.
- **Switch** : Commutateur de niveau 2 gérant la table d'adresses MAC et la transmission efficace au sein des réseaux locaux (VLANs).
- **Firewall (Pare-feu)** : Couche de sécurité analysant et filtrant le flux entrant et sortant selon des listes de règles définies.
- **Serveur** : Héberge des applications ou services et répond aux requêtes de paquets (ex: HTTP, DNS).
- **Point d'accès Wi-Fi** : Pont de connectivité sans fil caractérisé par un SSID et un canal de transmission.
- **Terminal** : Postes clients de destination ou sources émettrices du trafic (ex: PC, smartphones).

2. Gestion de la Topologie et Interconnexions

La topologie du réseau est gérée par la classe `Topologie`, agissant comme une matrice d'objets. Les liens physiques reliant deux équipements sont matérialisés par la classe `Lien`, qui porte les attributs de performance réseau essentiels tels que la bande passante (Mbps) et la latence (ms).

3. Simulation du trafic et algorithme de routage

Le simulateur de trafic repose sur deux classes complémentaires : **Paquet** et **Simulateur**.

La classe **Paquet** modélise une unité de données circulant dans le réseau. Elle encapsule cinq attributs privés : l'adresse IP source, l'adresse IP destination, le protocole (TCP, UDP ou ICMP), la taille en octets et le niveau de priorité (1 à 5). Ces attributs sont accessibles uniquement via des getters, conformément au principe d'encapsulation.

La classe **Simulateur** constitue le cœur algorithmique du projet. À partir de la topologie existante, elle construit un graphe interne où chaque équipement est un nœud et chaque lien une arête pondérée par la latence. C'est sur ce graphe qu'opère l'algorithme de routage.

Choix de l'algorithme : Dijkstra

Deux algorithmes étaient envisageables : BFS et Dijkstra. BFS trouve le chemin avec le moins de sauts mais ignore les caractéristiques des liens. Dijkstra trouve le chemin dont le coût total en latence est minimal, ce qui correspond directement à la métrique principale de SIMNet : le temps de transit simulé. Ce choix garantit que chaque paquet emprunte toujours le chemin le plus rapide, et non simplement le plus court en nombre de sauts. L'implémentation utilise le module **heapq** de la bibliothèque standard Python, qui fournit une file de priorité permettant d'explorer en priorité le nœud le moins coûteux. La méthode **__dijkstra()** est privée elle n'est accessible que depuis **envoyer_paquet()**, garantissant que tout routage passe par la validation complète du paquet.

4. Sécurité et filtrage

Le cœur algorithmique réside dans la classe **Simulateur**. Elle calcule les chemins logiques entre les terminaux, orchestre le routage des instances de la classe **Paquet**, et applique les règles d'inspection de la classe **RegleFiltrage** (AND logique sur les pages CIDR, ports et protocoles).

5. Surveillance et rapports

La supervision du réseau est assurée par la classe **Moniteur**, définie dans **moniteur.py**. Elle reçoit en paramètres une instance de **Topologie** et une instance de **Simulateur**, dont elle exploite les données sans les modifier respectant ainsi le principe de séparation des responsabilités.

Le moniteur maintient deux structures de données : un dictionnaire **stats_eq** comptabilisant pour chaque équipement le nombre de paquets transmis et perdus, et un dictionnaire **utilisation_liens** mesurant le volume de données en octets ayant transité sur chaque lien. Ces structures utilisent **defaultdict** du module **collections**, ce qui évite toute initialisation explicite des compteurs.

À la demande, le moniteur affiche un tableau de bord complet en console listant les équipements actifs et inactifs, l'utilisation des liens et l'historique des dix derniers paquets. Il peut également générer un fichier texte **rapport_simnet.txt** horodaté contenant l'ensemble de ces métriques, exportable à tout moment depuis le menu principal.

II. CONCEPTS POO COUVERTS

La conception de SIMNet a mobilisé l'ensemble des concepts fondamentaux de la programmation orientée objet enseignés dans ce cours.

Abstraction. La classe **Equipement** hérite de **abc.ABC** et déclare **afficher_info()** comme méthode abstraite via le décorateur **@abstractmethod**. Il est donc impossible d'instancier directement un objet **Equipement** seules ses sous-classes concrètes peuvent l'être. Ce mécanisme force chaque type d'équipement à implémenter sa propre version de **afficher_info()**.

Héritage. Les six classes **Routeur**, **Switch**, **Serveur**, **Firewall**, **PointAccesWifi** et **Terminal** héritent toutes de **Equipement**. Chacune appelle **super().__init__()** pour initialiser les attributs communs, puis ajoute ses propres attributs spécifiques. Ce mécanisme évite toute duplication de code.

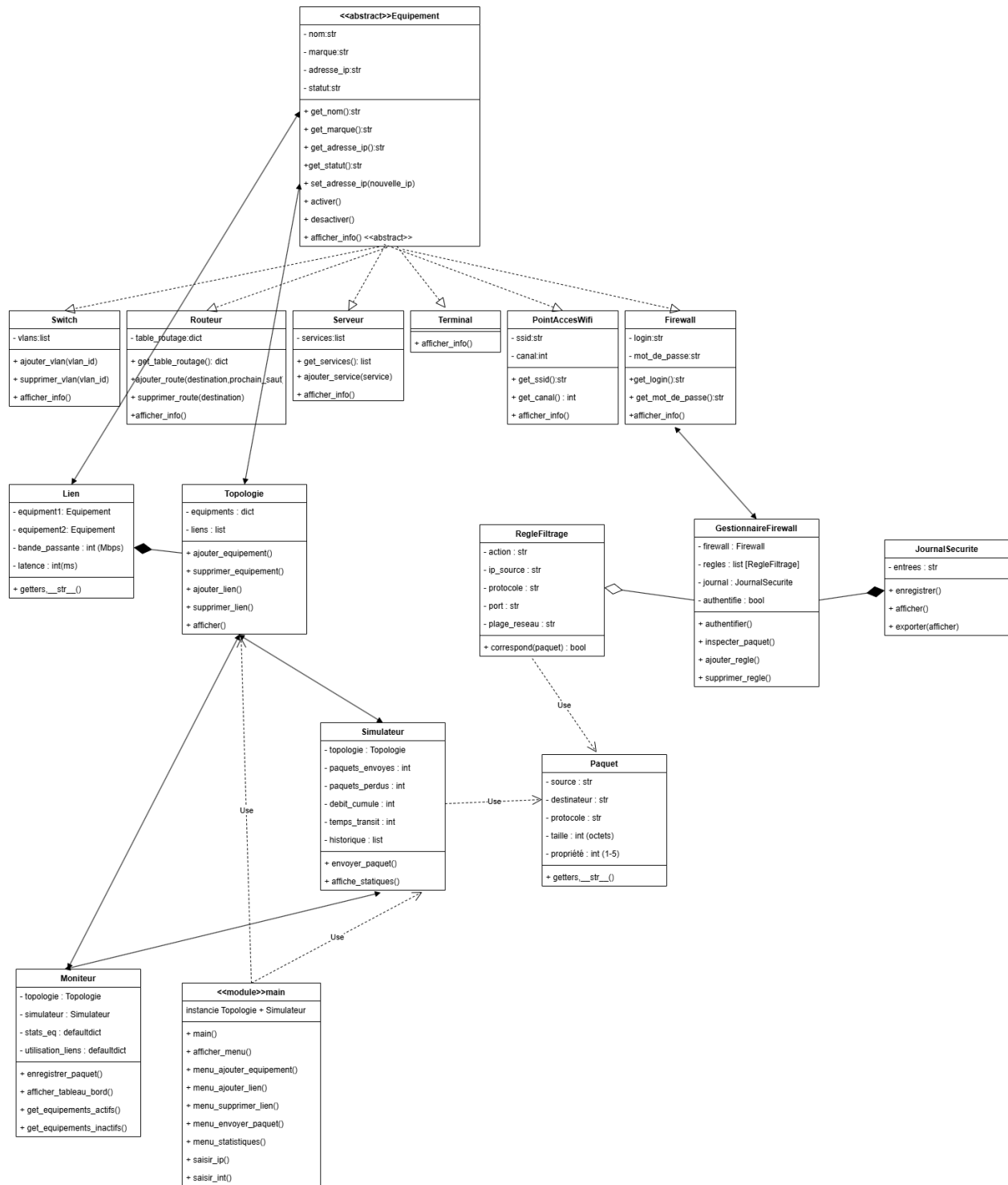
Encapsulation. Tous les attributs de toutes les classes sont déclarés privés avec le préfixe double underscore (**__nom**, **__adresse_ip**, **__regles**, etc.). L'accès extérieur se fait exclusivement via des getters. Certaines méthodes algorithmiques sensibles comme **__dijkstra()** et **__construire_graphe()** dans **Simulateur** sont également privées.

Polymorphisme. La méthode **afficher_info()** est redéfinie différemment dans chacune des six sous-classes d'**Equipement**. Appeler cette méthode sur n'importe quel objet du réseau produit un affichage adapté à son type, sans que le code appelant ait besoin de connaître la nature exacte de l'équipement.

Composition. La classe **Topologie** contient des instances de **Lien** dans sa liste **__liens**. Ces liens n'ont pas d'existence indépendante de la topologie si la topologie est détruite, les liens le sont aussi. De même, **GestionnaireFirewall** crée et possède un objet **JournalSecurite** à l'instanciation.

Agrégation. **GestionnaireFirewall** maintient une liste de **RegleFiltrage**. Contrairement à la composition, ces règles peuvent être ajoutées et supprimées indépendamment du gestionnaire — elles ont une existence propre.

III. DIAGRAMME DE CLASSES COMPLET



COMMENTAIRE :

Le diagramme ci-dessus représente l'architecture complète de SIMNet selon les conventions UML standard.

Au sommet de la hiérarchie se trouve la classe abstraite **Equipement**, dont héritent les six classes concrètes d'équipements réseau. Cette relation d'héritage est représentée par une ligne pleine surmontée d'un triangle vide pointant vers la classe parente. Elle traduit le principe de généralisation : tout équipement du réseau partage un socle commun d'attributs et de comportements, tout en spécialisant sa propre logique via **afficher_info()**.

La classe **Topologie** entretient deux relations structurelles : une composition avec **Lien** (losange plein), signifiant qu'un lien ne peut exister sans la topologie qui le contient, et une association avec **Equipement** (flèche ouverte), car elle référence les équipements sans en être propriétaire.

La classe **Simulateur** est associée à **Topologie** dont elle dépend pour construire le graphe de routage, et dépend de **Paquet** (ligne en tirets) qu'elle utilise temporairement lors de chaque simulation sans en assumer la création.

Le module de sécurité repose sur trois classes collaborantes. **GestionnaireFirewall** compose un **JournalSecurite** (losange plein — le journal lui appartient) et agrège des **RegleFiltrage** (losange vide — les règles peuvent être ajoutées et supprimées indépendamment). Il est également associé à **Firewall** dont il exploite les credentials d'authentification. **RegleFiltrage** dépend de **Paquet** pour sa méthode **correspond()**.

Enfin, **Moniteur** est associé à **Topologie** et **Simulateur** dont il lit les données pour produire tableaux de bord et rapports.

IV. EXTRAITS DU CODE SOURCE

Classe abstraite **Equipement** encapsulation et abstraction

```
from abc import ABC, abstractmethod

class Equipement(ABC):
    def __init__(self, nom, marque, adresse_ip):
        self.__nom = nom
        self.__marque = marque
        self.__adresse_ip = adresse_ip
        self.__statut = "actif"

    def get_nom(self):
        return self.__nom

    def est_actif(self):
        return self.__statut == "actif"

    @abstractmethod
    def afficher_info(self):
        pass
```

Ce code illustre l'abstraction via **abc.ABC** et **@abstractmethod**, l'encapsulation via les attributs privés **__nom**, **__marque**, **__adresse_ip**, et l'héritage via la méthode **super().__init__()** appelée dans chaque sous-classe.

Algorithme de Dijkstra **Simulateur**

```
def __dijkstra(self, source, destination):  
    graphe = self.__construire_graphe()  
    distances = {noeud: float('inf') for noeud in graphe}  
    distances[source] = 0  
    predecesseurs = {noeud: None for noeud in graphe}  
    file = [(0, source)]  
    while file:  
        dist_actuelle, noeud_actuel = heapq.heappop(file)  
        if noeud_actuel == destination:  
            break  
        for poids, voisin in graphe[noeud_actuel]:  
            nouvelle_dist = dist_actuelle + poids  
            if nouvelle_dist < distances[voisin]:  
                distances[voisin] = nouvelle_dist  
                predecesseurs[voisin] = noeud_actuel  
                heapq.heappush(file, (nouvelle_dist, voisin))  
    chemin = []  
    noeud = destination  
    while noeud is not None:  
        chemin.insert(0, noeud)  
        noeud = predecesseurs[noeud]  
    if len(chemin) == 0 or chemin[0] != source:  
        return None  
    return chemin
```

Ce code illustre l'encapsulation (méthode privée **__dijkstra**), l'utilisation du module standard **heapq**, et la reconstruction du chemin optimal par remontée des prédécesseurs.

Inspection de paquet **GestionnaireFirewall**

```
def inspecter_paquet(self, paquet):
    for regle in self.__regles:
        if regle.correspond(paquet):
            action = regle.get_action()
            self.__journal.enregistrer(action, paquet,
                                      raison=f"Regle: {regle}")

            return action
    self.__journal.enregistrer("AUTORISER", paquet,
                              raison="Aucune regle applicable")
    return "AUTORISER"
```

Ce code illustre le polymorphisme via **regle.correspond(paquet)** qui s'adapte à chaque règle, la composition via **self.__journal** qui appartient au gestionnaire, et l'algorithme first-match où la première règle correspondante s'applique immédiatement.

V. SCÉNARIOS DE CAS D'UTILISATION ET VALIDATION DU LOGICIEL

1. Scénario 1 : Initialisation de l'Architecture Réseau

Lors du lancement de SIMNet, l'administrateur fait face à un menu interactif lui permettant de bâtir sa topologie. Le programme applique un blindage strict sur les entrées utilisateur :

Contraintes d'adresses : Toute saisie d'adresse IP est validée par un algorithme vérifiant le format standardisé IPv4 (4 octets compris entre 0 et 255).

Sécurisation des accès sans fil et d'administration : Lors de la création d'un Point d'accès Wi-Fi ou d'un Firewall, le système rejette les SSID ou Logins trop courts ou exclusivement numériques afin de simuler des exigences industrielles de sécurité. Le mot de passe du pare-feu requiert un minimum obligatoire de 6 caractères.

2. Scénario 2 : Simulation de Routage Nominal

Une fois la topologie établie et interconnectée, l'utilisateur initie l'envoi d'un paquet.

Le moteur de simulation analyse le graphe du réseau. Si un chemin physique valide existe, le paquet traverse les équipements un à un.

Le programme calcule et cumule la latence globale du trajet et déduit la vitesse de transmission théorique en fonction de la bande passante la plus faible du parcours (goulot d'étranglement). Les statistiques de réussite sont immédiatement incrémentées dans le tableau de bord global du simulateur.

3. Scénario 3 : Comportement face aux Anomalies et Sécurité

Le logiciel a été spécifiquement blindé pour résister aux mauvaises manipulations des utilisateurs sans jamais s'interrompre :

Équipements inexistantes ou introuvables : Si l'utilisateur saisit une IP source ou une machine de destination qui n'a pas été préalablement créée dans la topologie, le simulateur intercepte dynamiquement l'erreur logicielle (**KeyError**). Au lieu d'un plantage du terminal, le programme affiche une alerte contextuelle claire indiquant l'absence de la cible et retourne proprement au menu principal.

Rupture de faisceau ou isolation : Si un lien réseau est supprimé ou si le pare-feu rejette le paquet suite à une règle de filtrage correspondante (blocage d'un protocole spécifique comme ICMP), le simulateur affiche l'échec de la transmission et génère un rapport d'anomalie réseau dans les statistiques.

VI. DIFFICULTÉS RENCONTRÉS ET SOLUTIONS APPORTÉES

Conflits Git et travail collaboratif

Le travail simultané de cinq développeurs sur un même dépôt a généré des conflits Git, notamment sur les fichiers **main.py** et **securite.py** qui ont subi plusieurs révisions majeures en cours de semaine. La résolution a nécessité une coordination manuelle : identification des marqueurs de conflit <<<<<< **HEAD** et >>>>>> **group_2**, sélection du code à conserver, puis commit de résolution. Cette expérience a renforcé notre discipline de commit — chaque membre travaillant désormais exclusivement sur son fichier attribué avant tout **git pull**.

Problèmes d'harmonisation entre modules

L'intégration des cinq modules a révélé des incompatibilités d'interface : le **Moniteur** accédait initialement aux attributs privés du **Simulateur** via le name mangling Python (**_Simulateur_historique**), ce qui constituait une violation de l'encapsulation. La solution a été de systématiser les getters publics dans **Simulateur** et d'uniformiser les signatures de méthodes entre modules lors d'un point d'équipe en milieu de semaine.

Gestion du délai et organisation de l'équipe

La durée d'une semaine a imposé une discipline rigoureuse. Les premiers jours ont été consacrés à la conception commune du diagramme de classes, ce qui a évité des refactorisations coûteuses en fin de projet. Malgré cela, certaines fonctionnalités ont dû être simplifiées pour respecter la deadline notamment l'intégration complète du firewall dans le flux d'envoi de paquets, qui a été partiellement externalisée dans **main.py**. Cette contrainte nous a appris à prioriser les livrables fonctionnels sur les livrables parfaits.

Indentation et syntaxe Python

Python étant sensible à l'indentation, plusieurs erreurs **IndentationError** sont apparues lors de la fusion de code rédigé par des membres utilisant des éditeurs différents (mélange espaces/tabulations). La solution adoptée a été de configurer VS Code pour forcer les espaces sur tous les postes et d'utiliser **python src/main.py** comme test de validation systématique avant chaque commit.

CONCLUSION ET PERSPECTIVES

SIMNet est le fruit d'une semaine de travail collaboratif intense, au cours de laquelle notre groupe a mobilisé l'ensemble des concepts fondamentaux de la programmation orientée objet en Python. De la conception du diagramme de classes initial jusqu'à l'intégration finale des cinq modules, chaque étape a été l'occasion de confronter la théorie à la réalité du développement en équipe.

Le simulateur livré répond aux exigences du cahier des charges : une modélisation orientée objet rigoureuse des équipements réseau, un moteur de routage basé sur l'algorithme de Dijkstra, un système de filtrage par règles avec journalisation horodatée, une surveillance des métriques réseau et une interface console interactive et robuste.

Au-delà du code, ce projet nous a enseigné autant sur le travail en équipe que sur Python lui-même — la rigueur des commits Git, la discipline des interfaces entre modules et la valeur d'une conception pensée en amont plutôt que corrigée en aval.

En termes de perspectives, SIMNet pourrait être enrichi par le développement d'une interface graphique, l'intégration de protocoles de routage réels tels qu'OSPF ou RIP, ou encore la simulation de pannes réseau dynamiques. Ces évolutions confirmeraient la solidité de l'architecture orientée objet choisie, conçue dès le départ pour être extensible.

BIBLIOGRAPHIE

Documentation officielle

- Python Software Foundation. *Python 3 Documentation abc : Abstract Base Classes*. Disponible sur : <https://docs.python.org/3/library/abc.html>
- Python Software Foundation. *Python 3 Documentation heapq : Heap queue algorithm*. Disponible sur : <https://docs.python.org/3/library/heapq.html>
- Python Software Foundation. *Python 3 Documentation collections : defaultdict*. Disponible sur : <https://docs.python.org/3/library/collections.html>
- Python Software Foundation. *Python 3 Documentation ipaddress : IPv4/IPv6 manipulation library*. Disponible sur : <https://docs.python.org/3/library/ipaddress.html>
- Python Software Foundation. *Python 3 Documentation datetime*. Disponible sur : <https://docs.python.org/3/library/datetime.html>

Ouvrages de référence

- Lutz, M. (2013). *Learning Python* (5e éd.). O'Reilly Media.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Algorithmes et réseaux

- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
- Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5e éd.). Pearson.